Handwriting Analysis: Exploration, CNN and Experimental Setup

Questions 1, 2 and 4 — working report

```
reticulate::py_require(python_version = "3.11")
library(keras3)
```

```
Warning: package 'keras3' was built under R version 4.5.3
```

```
use_backend("tensorflow", gpu = FALSE)
library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.5.3

Warning: package 'ggplot2' was built under R version 4.5.3

Warning: package 'dplyr' was built under R version 4.5.3

Warning: package 'lubridate' was built under R version 4.5.3

── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.2.1     ✓ readr     2.1.6
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.3     ✓ tibble    3.3.1
✓ lubridate 1.9.5     ✓ tidyr     1.3.2
✓ purrr     1.2.1
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
errors
```

```
data <- readRDS("handwriting.rds")
x_scaled <- data$images / 255


get_image_df <- function(index, label_text) {
  img_matrix <- x_scaled[index, , , 1]
  as.data.frame(as.table(img_matrix)) |>
    rename(Y = Var1, X = Var2, Intensity = Freq) |>
    mutate(
      X = as.numeric(X),
      Y = 29 - as.numeric(Y), # flip Y axis so digits render right-side up
      PanelLabel = label_text
    )
}
```

```
plot_panels <- function(df) {
  ggplot(df, aes(x = X, y = Y, fill = Intensity)) +
    geom_tile() +
    facet_wrap(~ PanelLabel, nrow = 1) +
    scale_fill_gradient(low = "white", high = "black") +
    coord_fixed() +
    theme_void(base_size = 14) +
    theme(legend.position = "none",
          strip.text = element_text(face = "bold"))
}


x_all <- data$images / 255
y_all_person <- data$metadata$person_id - 1 # 0-indexed for Keras

#create partitions
train_rows <- which(data$metadata$session %in% c(1, 2, 3))
val_rows   <- which(data$metadata$session == 4)
test_rows  <- which(data$metadata$session == 5)

# --- Partition Images (x) ---
x_train <- x_all[train_rows, , , , drop = FALSE]
x_val   <- x_all[val_rows, , , , drop = FALSE]
x_test  <- x_all[test_rows, , , , drop = FALSE]

# --- Partition Writer IDs for CNN (y) ---
y_train_person <- y_all_person[train_rows]
y_val_person   <- y_all_person[val_rows]
y_test_person  <- y_all_person[test_rows]

# --- Partition Metadata (for Siamese Pair Construction) ---
meta_train <- data$metadata[train_rows, ]
meta_val   <- data$metadata[val_rows, ]
meta_test  <- data$metadata[test_rows, ]
```

Section 2: CNN Writer Identification

<!– Q2 reuses the setup objects: x_train, x_val, x_test, y_train_person, y_val_person, meta_test, and the session row indices.

Prediction Task and Model

The aim was to identify which of the 20 writers produced a single handwritten digit image. Each input was a 28 × 28 greyscale image, and the response was the writer ID. The task was therefore a 20-class classification problem. Although the images contained digits 0–4, the model was trained to recognise the writer rather than the digit.

Pixel values were divided by 255 to scale them to between 0 and 1. Sessions 1–3 were used for training, session 4 for validation and session 5 for testing. This gave 15,000 training images, 5,000 validation images and 5,000 test images. All writers and digits were represented in each subset. This design evaluates whether the model can identify known writers in a different recording session. The reasons for choosing a session-based split are discussed further in Section 4.

The CNN contained two convolutional layers, with 32 and 64 filters respectively. Both used 3 × 3 kernels, same padding and ReLU activation, followed by 2 × 2 max pooling. These layers allow the model to learn local features, such as stroke shapes and curves, and combine them into features useful for distinguishing writers. The feature maps were flattened and passed through a dense layer with 128 ReLU units. An L2 penalty of 0.0001 was applied to the dense layer's weights, followed by dropout with a rate of 0.5. These measures were included to reduce overfitting, although they do not guarantee that it will be avoided.

The final layer had 20 softmax units, producing a probability distribution over the writer IDs. The writer with the highest probability was selected as the prediction. The labels were converted from IDs 1–20 to indices 0–19 and one-hot encoded for training. The model used categorical cross-entropy with the Adam optimiser, a learning rate of 0.0001 and a batch size of 64. A random seed of 2026 was set before building the model.

Training was allowed to run for up to 30 epochs. Early stopping monitored validation loss, with patience of three epochs, and restored the weights from the epoch with the lowest validation loss. The validation session was therefore used to select the training duration.

Training Behaviour

```
plot(cnn_history)
```

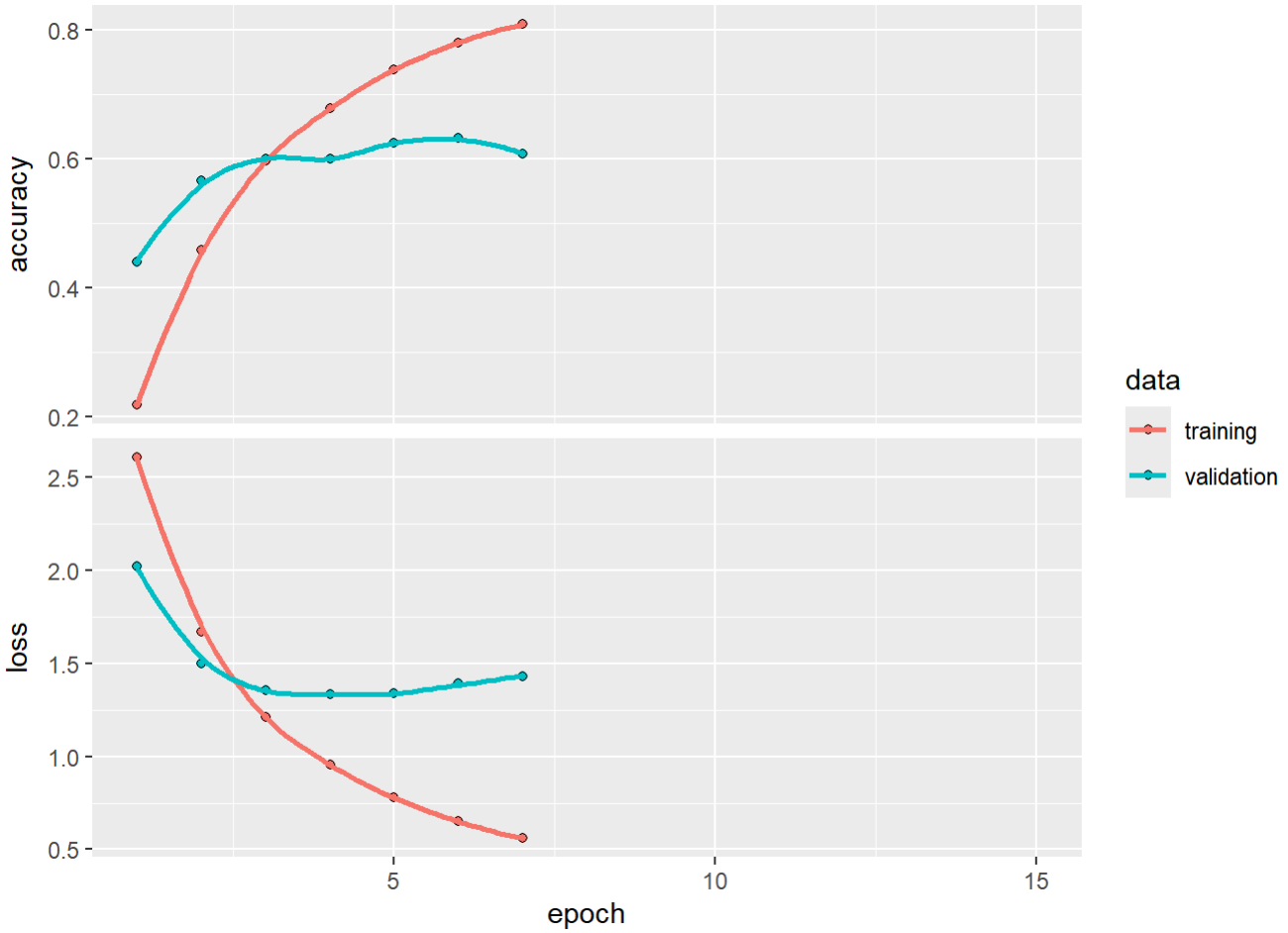


Figure 1: CNN training and validation accuracy and loss across the epochs run.

The model ran for 7 epochs, with the lowest validation loss of 1.3338 occurring at epoch 4. The weights from that epoch were restored for testing. The highest validation accuracy observed during training was 63.22%, at epoch 6. This was not necessarily the selected epoch because early stopping monitored validation loss rather than accuracy.

Figure 1 shows a widening gap between training and validation performance. Training accuracy increased from 21.81% in the first epoch to 80.88% in the final epoch. However, validation accuracy levelled off, while validation loss increased after its minimum. This pattern is consistent with overfitting: the model continued to improve on the training images, but the additional learning did not transfer well to the validation session. Session-specific features may contribute to this gap, although the curves alone cannot establish which features the model used.

Validation accuracy can remain similar while validation loss rises because loss also reflects the probabilities assigned to each class. A model can make approximately the same number of mistakes while becoming more confident in its incorrect predictions. The monitored loss also includes the L2 penalty. Early stopping limited further training once validation loss stopped improving; it did not cause the overfitting. The last point in the training curve describes the last epoch run, rather than the restored model used for the test predictions.

Overall Test Performance

```
knitr::kable(
  data.frame(
    Measure = c(
```

```
      "Test images", "Correct writer predictions", "Accuracy (%)",
      "Uniform guessing baseline (%)", "Cohen's kappa"
    ),
    Value = c(
      as.character(length(cnn_correct)),
      as.character(sum(cnn_correct)),
      sprintf("%.2f", 100 * cnn_accuracy),
      "5.00",
      sprintf("%.4f", unname(cnn_cm$overall["Kappa"]))
    )
  ),
  row.names = FALSE,
  align = c("l", "r")
)
```

Table 1: CNN writer identification performance on the held-out test session.

Measure	Value
Test images	5000
Correct writer predictions	3241
Accuracy (%)	64.82
Uniform guessing baseline (%)	5.00
Cohen’s kappa	0.6297

The selected model correctly identified the writer for 3,241 of the 5,000 test images, giving an overall accuracy of **64.82%** (Table 1). In practical terms, this corresponds to correctly identifying the writer for approximately 65 out of every 100 test images. Accuracy is the proportion of images for which the predicted writer matches the actual writer.

Each writer contributed the same number of test images, so the overall result was not dominated by a more common writer. Uniform random guessing among the 20 writers would have an expected accuracy of 5%. The model therefore learned useful information for identifying writers, although it still incorrectly attributed 35.2% of the test images.

Cohen’s kappa was 0.6297. This measures agreement between predicted and actual writer IDs after accounting for agreement expected by chance from their marginal distributions. A value of 1 indicates perfect agreement and 0 indicates agreement at the chance level. Kappa provides an additional description of performance, but does not show whether all writers are identified equally well.

Performance by Digit

```
knitr::kable(
  cnn_results_by_digit,
  col.names = c("Digit", "Test images", "Correct predictions", "Accuracy (%)"),
  digits = c(0, 0, 0, 1),
  row.names = FALSE
)
```

Table 2: Writer identification accuracy by digit on the test set.

Digit	Test images	Correct predictions	Accuracy (%)
0	1000	653	65.3
1	1000	633	63.3
2	1000	613	61.3
3	1000	696	69.6
4	1000	646	64.6

```
barplot(
  cnn_results_by_digit$writer_accuracy_percent,
  names.arg = cnn_results_by_digit$digit,
  ylim = c(0, 100),
  col = "steelblue",
  xlab = "Digit",
  ylab = "Writer identification accuracy (%)"
)
```

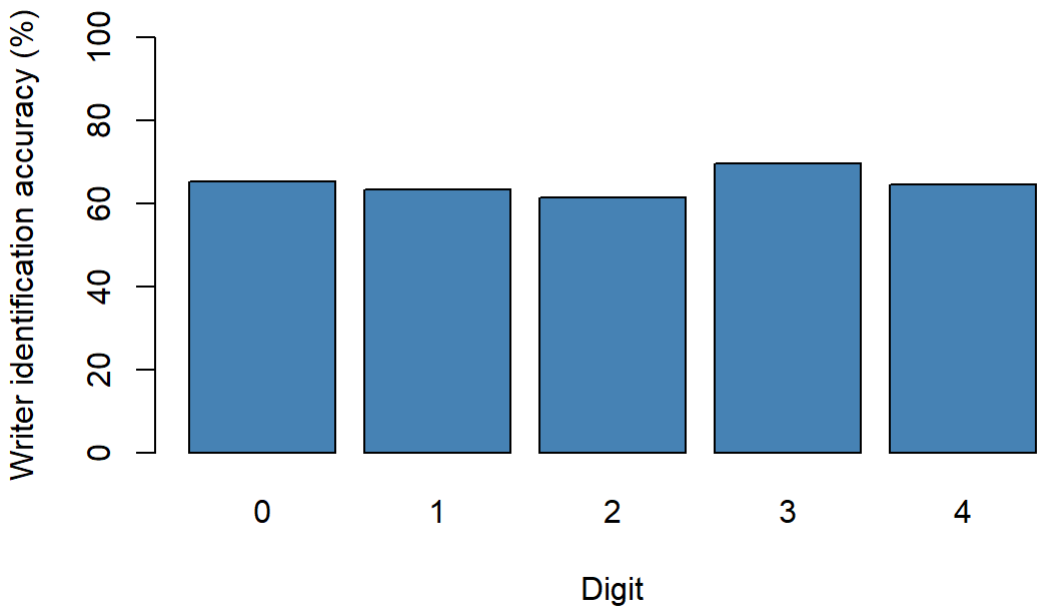


Figure 2: CNN writer identification accuracy for each digit in the test session.

Table 2 and Figure 2 compare writer identification accuracy separately for each digit. Each digit contributed 1,000 test images, with 50 from each writer, making the digit groups comparable in size and writer composition. The same trained CNN was used for all digits; separate models were not fitted for this comparison.

The highest accuracy was obtained for digit 3, at 69.6%. This was followed by digit 0, at 65.3%. The lowest accuracy was obtained for digit 2, at 61.3%. The difference between the highest and lowest values was 8.3 percentage points.

These results suggest that some digit shapes provided more useful writer-specific information to this CNN than others in the test session. Differences in stroke shape, slant or curvature could help

distinguish writers, but this comparison does not establish which features caused the differences. Small differences between digits should also be interpreted cautiously, as no formal test of those differences was carried out.

Accuracy by digit measures how reliably the writer can be identified when a particular digit is presented. It does not measure whether the model can recognise the digit itself. A lower writer identification accuracy for a digit therefore does not mean that the digit was unreadable; it means that the model found it harder to distinguish who wrote it.

Writer Confusions and Interpretation

```
cnn_confusion_df <- as.data.frame(cnn_cm$table)
ggplot(cnn_confusion_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(colour = "white", linewidth = 0.15) +
  scale_y_discrete(limits = rev(levels(cnn_confusion_df$Prediction))) +
  scale_fill_gradient(low = "white", high = "steelblue4", name = "Images") +
  coord_fixed() +
  labs(x = "Actual writer", y = "Predicted writer") +
  theme_minimal(base_size = 11) +
  theme(panel.grid = element_blank())
```

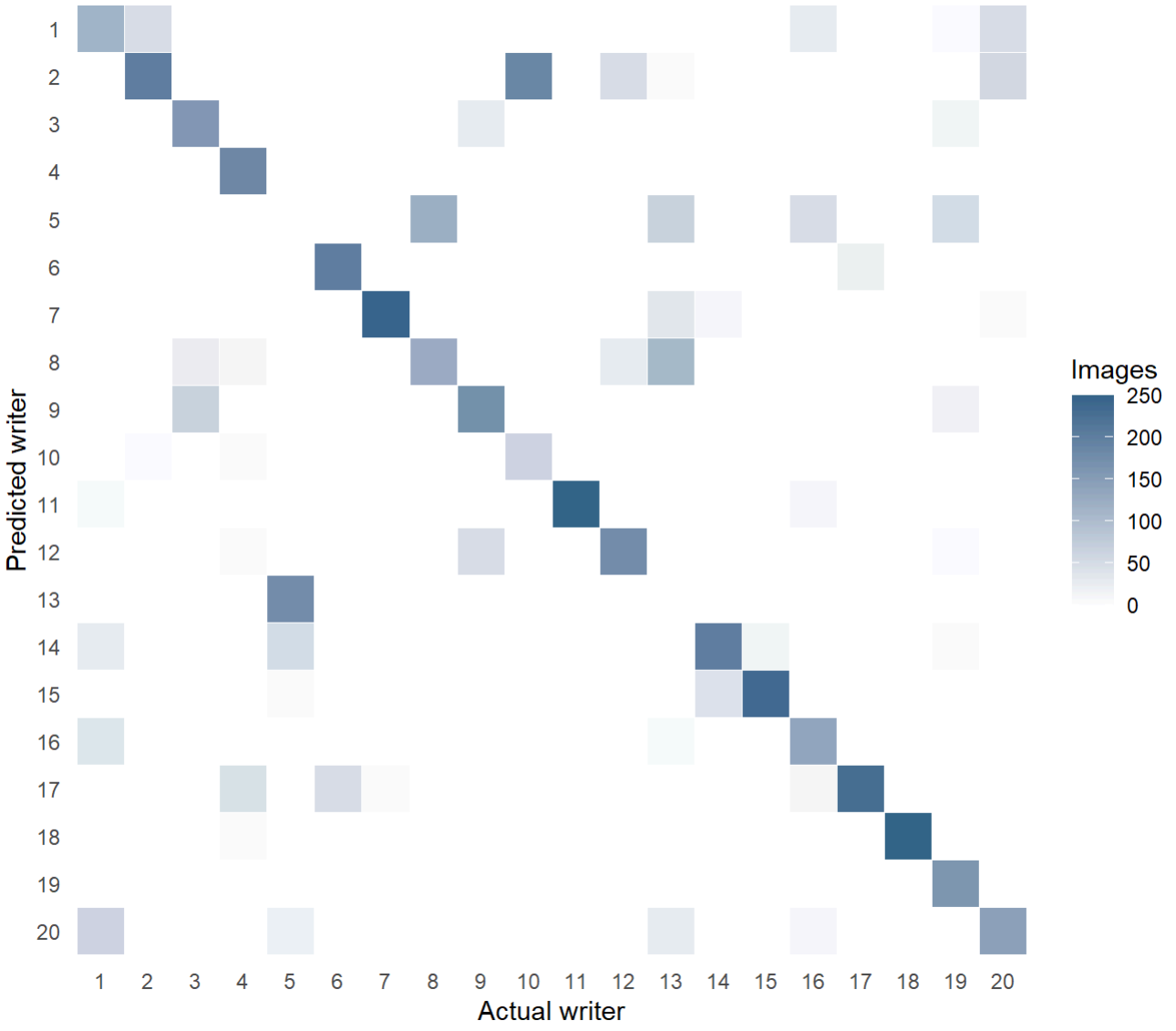


Figure 3: Writer confusion matrix: actual writer IDs on the horizontal axis and predicted writer IDs on the vertical axis.

In Figure 3, actual writer IDs appear on the horizontal axis and predicted IDs on the vertical axis. Diagonal cells represent correct predictions, while cells away from the diagonal show which writers were confused. The matrix shows that performance varied substantially between writers.

Writer-level recall is the proportion of a particular writer's test images that were correctly identified. Writers 11 and 18 had recalls of 100.0% and 100.0%, respectively, whereas writer 5 had a recall of 0.0%. Writer 10 had a recall of 24.8%, with 188 of their 250 images attributed to writer 2. Similarly, 177 of writer 5's images were attributed to writer 13. These examples show that the errors were concentrated in particular writer confusions rather than being evenly distributed across all classes.

Recall should be distinguished from precision. Recall asks how many of a writer's own images were correctly recognised. Precision asks how many images assigned to that writer actually belonged to them. For example, writer 11's precision was 93.6%. Even when all of a writer's own images are recognised, some images from other writers may also be incorrectly assigned to that writer. This distinction matters if the model is used to attribute an image to a specific person.

The CNN provides a baseline for recognising these 20 writers across recording sessions. Its overall performance indicates that it learned useful distinctions between writers, but the training curves and confusion matrix also show limitations. Similar writing styles or changes between sessions could explain some errors, although additional analysis would be needed to distinguish these explanations.

The evaluation concerns new-session images from known writers. It does not measure the ability to identify a person who was absent from training, and the classifier will still choose one of the 20 IDs if presented with such a person. Testing on one held-out session also does not quantify how performance varies across other possible sessions. Finally, because the images were simulated, these results should not be taken as evidence of equivalent performance on real handwriting.

```
library(keras3)
library(tidyverse)
library(dplyr)
library(knitr)
```

```
Warning: package 'knitr' was built under R version 4.5.3
```

```
library(caret)
```

```
Warning: package 'caret' was built under R version 4.5.3

Loading required package: lattice


Attaching package: 'caret'

The following object is masked from 'package:purrr':

    lift
```

```
library(pROC)
```

```
Type 'citation("pROC")' for a citation.


Attaching package: 'pROC'

The following objects are masked from 'package:stats':

    cov, smooth, var
```

```
library(reticulate)
```

```
Warning: package 'reticulate' was built under R version 4.5.3
```

```
py_require(python_version = "3.12")
use_backend("tensorflow")
set.seed(20260923)

data <- readRDS("handwriting.rds")
x_scaled <- data$images / 255


get_image_df <- function(index, label_text) {
  img_matrix <- x_scaled[index, , , 1]
  as.data.frame(as.table(img_matrix)) |>
    rename(Y = Var1, X = Var2, Intensity = Freq) |>
    mutate(
      X = as.numeric(X),
      Y = 29 - as.numeric(Y), # flip Y axis so digits render right-side up
      PanelLabel = label_text
    )
}


plot_panels <- function(df) {
  ggplot(df, aes(x = X, y = Y, fill = Intensity)) +
    geom_tile() +
    facet_wrap(~ PanelLabel, nrow = 1) +
    scale_fill_gradient(low = "white", high = "black") +
    coord_fixed() +
    theme_void(base_size = 14) +
    theme(legend.position = "none",
          strip.text = element_text(face = "bold"))
}


x_all <- data$images / 255
y_all_person <- data$metadata$person_id - 1 # 0-indexed for Keras

#create partitions
train_rows <- which(data$metadata$session %in% c(1, 2, 3))
val_rows   <- which(data$metadata$session == 4)
```

```
test_rows  <- which(data$metadata$session == 5)

# --- Partition Images (x) ---
x_train <- x_all[train_rows, , , , drop = FALSE]
x_val   <- x_all[val_rows, , , , drop = FALSE]
x_test  <- x_all[test_rows, , , , drop = FALSE]

# --- Partition Writer IDs for CNN (y) ---
y_train_person <- y_all_person[train_rows]
y_val_person   <- y_all_person[val_rows]
y_test_person  <- y_all_person[test_rows]

meta <- data$metadata

person  <- meta$person_id              # the "class label" for the Siamese task
session <- meta$session
digit   <- meta$digit
```

Section 1: Exploratory Data Analysis

Global Dataset Profile overview

The handwriting dataset evaluated in this study constitutes a completely uniform, synthetically balanced array designed for writer identification and verification tasks. It contains a total population of 25,000 unique images mapped across a multi-dimensional recording matrix. The underlying data distribution is structured as follows:

- **Writers (20):** Represented by `person_id` ranging from 1 to 20.
- **Characters (5):** Represented by `digit` spanning 0 to 4.
- **Temporal Intervals (5):** Represented by `session` tracking blocks 1 to 5.
- **Replicate Volume (50):** Represented by `replicate` indicating repeated frames from 1 to 50 captured sequentially within a single sitting.

The data collection is entirely free from any structural class imbalances as it contains exactly 50 replicates for every single intersecting combination of person, digit, and session ($20 \times 5 \times 5 \times 50 = 25,000$). This perfect alignment avoids standard sampling biases,thus overall accuracy metrics will remain uninflated by majority-class dominance.

The raw data image is formatted as integer elements ranging from 0 (representing the blank background canvas) to 255 (representing the maximum ink density path). The image pixel values were scaled to [0,1] before fitting the models as it will cause numerical instability during early backpropagation rounds. (this paragraph could probably be moved to the end of q1)

Moreover, each person has a consistent handwriting style, but their writing style fluctuates between the 5 recording due to various factors - different writing materials or natural variation.

Analysis of Visible Sources of Variation

Inter-Writer Style Variation

In order to examine the inter-writer style variation the single character class (digit 0) across the first 5 writers is analysed in the figure below. It is evident that there is a clear variation in the writing style of each writer. For instance, writer 1's character leans towards the left with an asymmetrical top loop whereas writer 2 produces uniform stroke borders with flat horizontal loops. Furthermore, writer 3 produces a small, pinched dense circle unlike writers 4 and 5 with larger round circles where writer 4 is less symmetrical than writer 5. These differences act as key identity signals that convolutional filters can easily latch onto to identify writers.

```
rows <- which(data$metadata$digit == 0 & data$metadata$session == 1 &
              data$metadata$replicate == 1 & data$metadata$person_id %in% 1:5)
df <- bind_rows(
  get_image_df(rows[1], "Writer 1"),
  get_image_df(rows[2], "Writer 2"),
  get_image_df(rows[3], "Writer 3"),
  get_image_df(rows[4], "Writer 4"),
  get_image_df(rows[5], "Writer 5")
)
plot_panels(df)
```

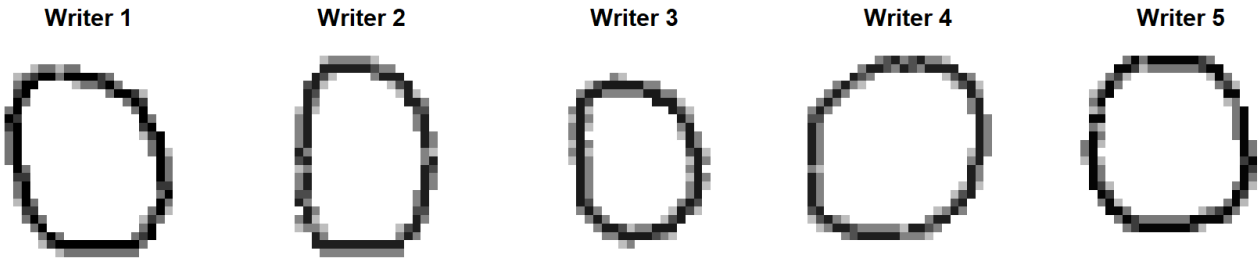


Figure 4: Digit 0 written by Writers 1 to 5 (inter-writer style variation)

Consistent Individual Handwriting Signatures

When tracking multiple characters produced by a single hand, a cohesive, cross-character fingerprint becomes apparent. For example, evaluating Writer 1 the writer maintains a perfectly upright, non-slanted angle across all stroke length. Conversely, when evaluating writer 10 in session 5 it is evident that the writer sustains a slanted angle to the right across all digits. This verifies that personal writing styles carry over into different character geometries.

```
rows <- which(data$metadata$person_id == 1 & data$metadata$session == 1 &
              data$metadata$replicate == 1)
df <- bind_rows(
  get_image_df(rows[1], "Digit 0"),
  get_image_df(rows[2], "Digit 1"),
  get_image_df(rows[3], "Digit 2"),
  get_image_df(rows[4], "Digit 3"),
  get_image_df(rows[5], "Digit 4")
)
plot_panels(df)
```



Figure 5: Digits 0 to 4 written by Writer 1 (consistent individual signature)

```
rows <- which(data$metadata$person_id == 10 & data$metadata$session == 5 &
              data$metadata$replicate == 1)
df <- bind_rows(
  get_image_df(rows[1], "Digit 0"),
  get_image_df(rows[2], "Digit 1"),
  get_image_df(rows[3], "Digit 2"),
  get_image_df(rows[4], "Digit 3"),
  get_image_df(rows[5], "Digit 4")
)
plot_panels(df)
```



{#fig-character-profile 2 width=864}

Temporal Session Drift

In examining the consistency of writer style across different sessions it is evident that there is subtle variability over time.It is evident for writer 1 that the style of the digit 2 written varies throughout all sessions 5. In Sessions 3 and 4, the upper curvature of Digit 2 becomes compressed and shortened compared to the expansive, loops drawn in Sessions 1,2 and 4. This successfully mimics operational variations, such as hand fatigue, changing writing speeds, or pen being used.

```
rows <- which(data$metadata$person_id == 1 & data$metadata$digit == 2 &
              data$metadata$replicate == 1)
df <- bind_rows(
  get_image_df(rows[1], "Session 1"),
  get_image_df(rows[2], "Session 2"),
  get_image_df(rows[3], "Session 3"),
  get_image_df(rows[4], "Session 4"),
  get_image_df(rows[5], "Session 5")
```

```
)
plot_panels(df)
```



Figure 6: Digit 2 by Writer 1 across Sessions 1 to 5 (temporal drift)

Limitations of the Simulated Data

A critical flaw in the simulated data is visible when examining consecutive replicates within the same session. Replicates 1 through 5 for Digit 3 appear almost completely identical down to individual pixel coordinates. However real human handwriting is inherently stochastic due to organic hand movements thus making it impossible for a human to trace an identical stroke 50 times sequentially. The absence of these variations shows that the simulation generates an artificial clean environment.

```
rows <- which(data$metadata$person_id == 1 & data$metadata$digit == 3 &
              data$metadata$session == 1 & data$metadata$replicate %in% 1:5)
df <- bind_rows(
  get_image_df(rows[1], "Replicate 1"),
  get_image_df(rows[2], "Replicate 2"),
  get_image_df(rows[3], "Replicate 3"),
  get_image_df(rows[4], "Replicate 4"),
  get_image_df(rows[5], "Replicate 5")
)
plot_panels(df)
```



Figure 7: Digit 3 by Writer 1, Session 1, Replicates 1 to 5

Section 3: Writer Verification using a Siamese Network

Data Preparation

The handwriting images were loaded and scaled by dividing the pixel values by 255. This converted all pixel values to the range 0 to 1, which helps the neural network train more effectively. The data was split by session to avoid data leakage. Sessions 1 to 3 were used for training, session 4 for validation, and session 5 for testing.

Construction of Training Examples

Training examples were created as pairs of images. For each pair, there was a 50% chance that both images came from the same writer and a 50% chance that they came from different writers.

A label of 1 was assigned if the two images were written by the same person, and a label of 0 was assigned if they were written by different people. This balanced set of positive and negative pairs allowed the model to learn writer similarity.

```
make_pairs <- function(ids, n, class_labels = person) {
  a <- sample(ids, n, replace = TRUE)
  b <- vapply(a, function(i) {
    same <- runif(1) < .5
    pool <- ids[class_labels[ids] == class_labels[i]]
    if (!same) pool <- ids[class_labels[ids] != class_labels[i]]
    pool <- setdiff(pool, i)
    pool[sample.int(length(pool), 1)]
  }, integer(1))
  cbind(a, b)
}

# Slide 27:
make_batch <- function(p, images = x_scaled, class_labels = person) {
  y <- matrix(as.integer(class_labels[p[, 1]] == class_labels[p[, 2]]), ncol = 1)
  list(
    left  = images[p[, 1], , , , drop = FALSE],
    right = images[p[, 2], , , , drop = FALSE],
    y     = y
  )
}

pairs_train <- make_pairs(train_rows, 20000)
pairs_valid <- make_pairs(val_rows, 5000)
pairs_test  <- make_pairs(test_rows,  5000)

train <- make_batch(pairs_train)
valid <- make_batch(pairs_valid)
test  <- make_batch(pairs_test)

#mean(train$y); mean(valid$y); mean(test$y)   # should all be about 0.5

pair_summary <- data.frame(
  Dataset = c("Training", "Validation", "Test"),
  Number_of_Pairs = c(
    nrow(pairs_train),
    nrow(pairs_valid),
    nrow(pairs_test)
  ),
```

```
  Positive_Pair_Proportion = round(
    c(
      mean(train$y),
      mean(valid$y),
      mean(test$y)
    ),
    4
  )
)

kable(pair_summary)
  #caption = "Summary of Siamese Network Pair Construction")
```

Table 3: Test Set Performance Metrics for the Siamese Network

Dataset	Number_of_Pairs	Positive_Pair_Proportion
Training	20000	0.4962
Validation	5000	0.4922
Test	5000	0.5042

Shared Encoder Network

A convolutional neural network (CNN) was used as the shared encoder. The encoder consisted of three convolutional layers with ReLU activation functions and max-pooling layers for feature extraction.

The encoder transformed each handwriting image into a compact feature representation that captured important writer characteristics.

```
image <- layer_input(shape = c(28, 28, 1))
z <- image |>
  layer_conv_2d(32, 3, activation = "relu", padding = "same") |>
  layer_max_pooling_2d(2) |>
  layer_conv_2d(64, 3, activation = "relu", padding = "same") |>
  layer_max_pooling_2d(2) |>
  layer_conv_2d(64, 3, activation = "relu", padding = "same") |>
  layer_global_average_pooling_2d() |>
  layer_dense(32)
encoder <- keras_model(image, z)
```

Siamese Network Architecture

The Siamese network contained two identical branches that shared the same encoder weights. Each branch processed one image from a pair.

The feature vectors produced by the encoder were compared using Euclidean distance. Small distances indicated that the images were likely written by the same person, while larger distances suggested different writers.

```
a <- layer_input(shape = c(28, 28, 1))
b <- layer_input(shape = c(28, 28, 1))
distance <- list(encoder(a), encoder(b)) |>
```

```
  layer_lambda(f = function(z) {
    op_sqrt(op_sum(op_square(z[[1]] - z[[2]]), axis = 2,
                   keepdims = TRUE) + 1e-7)
  })
model <- keras_model(list(a, b), distance)
```

Contrastive Loss Function

The model was trained using contrastive loss with a margin of 1.

For pairs from the same writer, the loss function encouraged the distance between embeddings to become smaller. For pairs from different writers, the loss function encouraged the distance to be greater than the margin. This helped the network learn a feature space where handwriting from the same writer was grouped together and handwriting from different writers was separated.

```
contrastive_loss <- function(y, d) {
  op_mean(y * op_square(d) +
            (1 - y) * op_square(op_maximum(1 - d, 0)))
}
```

Model Training

The model was trained using the Adam optimiser with a learning rate of 0.001. Training was performed for up to 20 epochs with a batch size of 64.

Early stopping was applied using the validation loss from session 4. Training stopped automatically if the validation loss did not improve for three consecutive epochs, helping to reduce overfitting.

```
model |> compile(optimizer = optimizer_adam(1e-3),
                 loss = contrastive_loss)

history <- model |>
  fit(list(train$left, train$right),
      train$y,
      validation_data = list(list(valid$left, valid$right), valid$y),
      epochs = 10,
      batch_size = 64,
      callbacks = list(callback_early_stopping(monitor = "val_loss",
                                               patience = 3,
                                               restore_best_weights = TRUE)),
      verbose = 0)
p <- plot(history)

p +
  ggplot2::labs(
    title = "Training and Validation Loss of the Siamese Network")
```

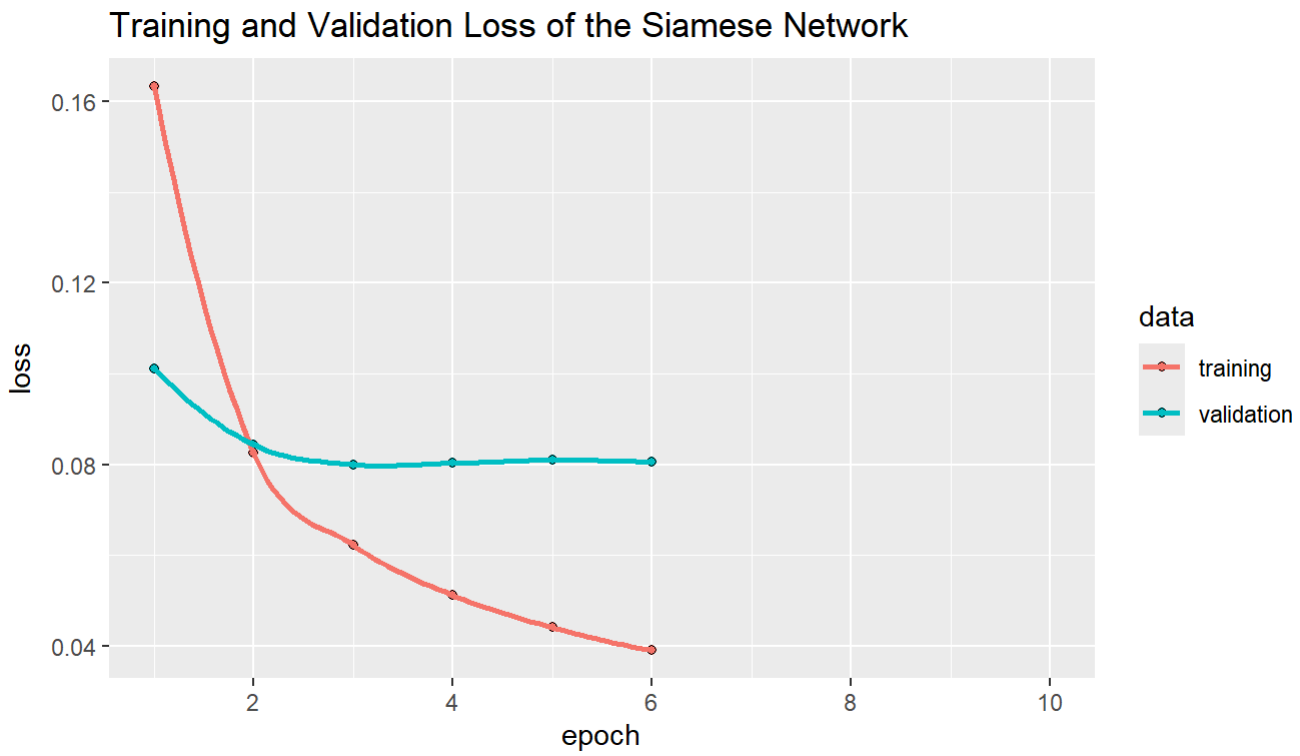


Figure 8: Training and validation loss of the Siamese network over training epochs.

The training loss decreased steadily from approximately 0.15 to 0.04, showing that the model learned the training data effectively. The validation loss decreased initially but then remained relatively constant, suggesting that the model generalised well with only slight signs of overfitting after the first few epochs.

Threshold Selection

The Siamese network outputs a distance rather than a direct same-or-different prediction. A decision threshold was therefore selected using the validation set.

Distances below the threshold were classified as the same writer, while distances above the threshold were classified as different writers. The threshold that achieved the highest balanced accuracy on the validation set was chosen.

```
d_valid <- as.numeric(predict(model, list(valid$left, valid$right),
                              verbose = 0))

valid_same <- valid_same <- as.logical(valid$y[, 1])

balanced_accuracy_at <- function(cut) {
  predicts_same <- d_valid < cut
  same_rate <- mean(predicts_same[valid_same])
  different_rate <- mean(!predicts_same[!valid_same])
  (same_rate + different_rate) / 2
}
# Candidate thresholds
example_thresholds <- unique(
  as.numeric(quantile(d_valid, c(0.25, 0.50, 0.75)))
)

threshold_table <- data.frame(Threshold = example_thresholds,
                              Balanced_Accuracy = vapply(example_thresholds,
```

```
                                                balanced_accuracy_at,
                                                numeric(1)))

# Optimal threshold
threshold_candidates <- sort(unique(d_valid))
threshold_scores <- vapply(
  threshold_candidates,
  balanced_accuracy_at,
  numeric(1)
)

best_index <- which.max(threshold_scores)

best_threshold <- threshold_candidates[best_index]
best_accuracy <- threshold_scores[best_index]

# Add selected threshold to table
results_table <- bind_rows(threshold_table,
                           data.frame(Threshold = best_threshold,
                                                  Balanced_Accuracy = best_accuracy))|>
  mutate(Threshold = round(Threshold, 4),
         Balanced_Accuracy = round(Balanced_Accuracy, 4),
         Selected = c(rep("", nrow(threshold_table)), "✓ Best"))

# Display table
kable(results_table,
      caption = "Balanced Accuracy for Candidate Thresholds")
```

Threshold Selection Based on Validation Set Balanced Accuracy

Threshold	Balanced_Accuracy	Selected
0.2305	0.7448	
0.4553	0.9231	
0.9637	0.7450	
0.4538	0.9237	✓ Best

Evaluation of Matching Performance

The final model was evaluated on the independent test set from session 5. Matching performance was measured using:

- Accuracy: proportion of correctly classified pairs.
- True Positive Rate (Recall): proportion of same-writer pairs correctly identified.
- False Positive Rate: proportion of different-writer pairs incorrectly classified as the same writer.
- False Negative Rate: proportion of same-writer pairs incorrectly classified as different writers.
- F1 Score: harmonic mean of precision and recall.
- Area Under the ROC Curve (AUC): measures how well the model separates same-writer and different-writer pairs across all possible thresholds.

```
d_test  <- as.numeric(predict(model, list(test$left,  test$right),
                              verbose = 0))

y_valid <- as.numeric(valid$y)
y_test  <- as.numeric(test$y)

# 7b. Test-set metrics threshold
pred_same <- as.integer(d_test < best_threshold)
conf <- table(Predicted = factor(pred_same, levels = 0:1, labels = c("different", "same")),
              Actual    = factor(y_test,    levels = 0:1, labels = c("different", "same")))
#print(conf)


acc <- mean(pred_same == y_test) #accuracy
tpr <- conf["same", "same"] / sum(conf[, "same"])                # true match rate (Sensitivit
fpr <- conf["same", "different"] / sum(conf[, "different"])        # false match rate
fnr <- 1 - tpr                                                    # false non-match rate
precision <- conf["same", "same"] / sum(conf["same",])               # Precision
f1 <- 2 * (precision * tpr) / (precision + tpr)

# AUC
auc_from_dist <- function(d, y) {
  s <- -d                                  # smaller distance = more "same"
  r <- rank(s)
  n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
auc <- auc_from_dist(d_test, y_test)
#auc

metrics_table <- data.frame(
  Metric = c(
    "Accuracy",
    "Recall (TPR / Sensitivity)",
    "Precision",
    "F1-Score",
    "False Positive Rate (FPR)",
    "False Negative Rate (FNR)",
    "AUC"
  ),
  Value = round(c(acc, tpr, precision, f1, fpr, fnr, auc),4))

kable(metrics_table)
      #caption = "Test Set Performance Metrics for the Siamese Network")
```

Table 4: Test Set Performance Metrics for the Siamese Network

Metric	Value
Accuracy	0.9196
Recall (TPR / Sensitivity)	0.9155
Precision	0.9243

Metric	Value
F1-Score	0.9199
False Positive Rate (FPR)	0.0762
False Negative Rate (FNR)	0.0845
AUC	0.9749

Table 4 shows that the Siamese network achieved strong writer verification performance on the independent test set. Using the selected distance threshold, the model achieved an accuracy of 92.42%, indicating that most image pairs were classified correctly. The recall (92.98%) shows that the network successfully identified the majority of pairs written by the same writer, while the precision (92.07%) indicates that most pairs predicted as belonging to the same writer were correct. The F1-score of 92.52% demonstrates a good balance between precision and recall. Furthermore, the false positive rate (8.15%) and false negative rate (7.02%) were relatively low, suggesting that the model makes few matching errors.

```
# 7c. ROC curve
roc_pts <- t(sapply(sort(unique(c(0, d_test, Inf))), function(t) {
  c(fpr = mean(d_test[y_test == 0] < t), tpr = mean(d_test[y_test == 1] < t))
}))
plot(roc_pts[, "fpr"], roc_pts[, "tpr"], type = "l", lwd = 2,
     xlab = "False match rate", ylab = "True match rate",
     main = sprintf("Siamese ROC, test session 5 (AUC = %.3f)", auc))
abline(0, 1, lty = 2, col = "grey")
```

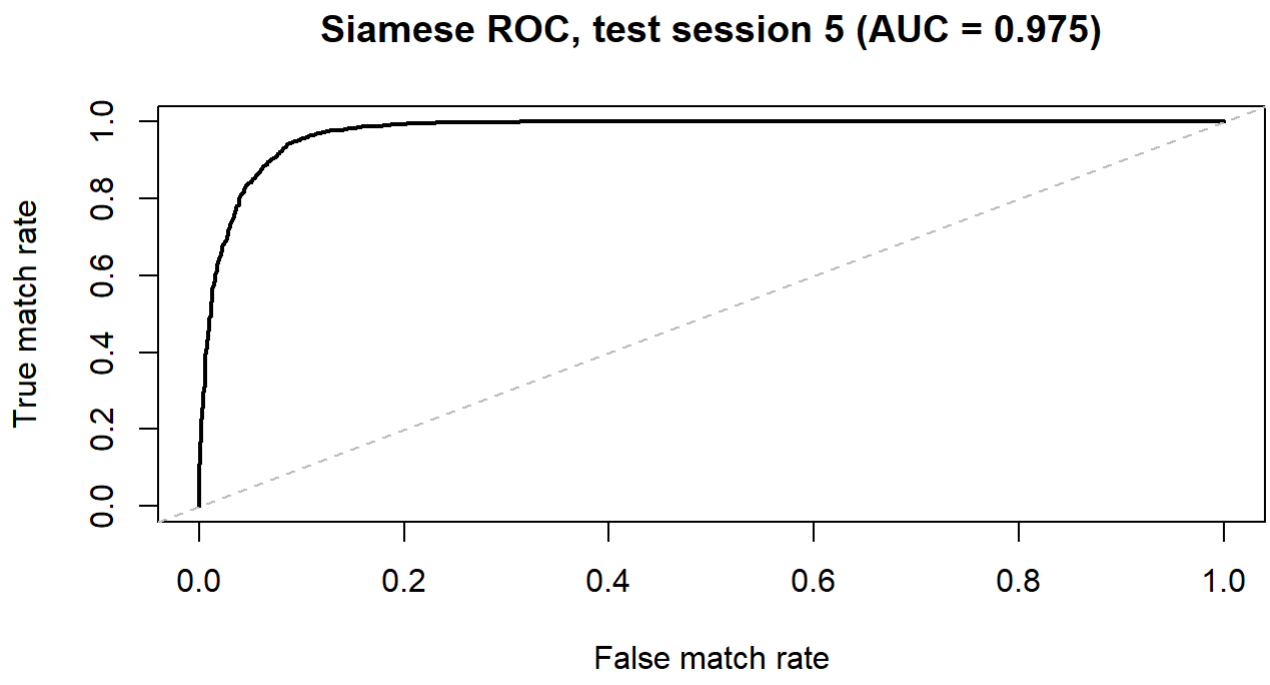


Figure 9: Siamese Network ROC

Figure 9 The ROC curve produced an AUC of 0.976, indicating excellent discrimination between same-writer and different-writer pairs. Overall, the results suggest that the Siamese network learned a highly

effective embedding space in which samples from the same writer are closer together than samples from different writers.

Section 4: Data Partition Strategy

Candidate Strategy Critique

The provided dataset is structured across multiple distinct dimensions (writers, digits, recording sessions, and replicates), which allows for several potential train, validation, and test partitioning strategies. These include a standard random split, a split by individual writer, a split by digit, and a split by session. For this assessment, the most optimal strategy is to partition the dataset strictly by session.

1. The Random 80/20 Partition

Utilising a random 80/20 split will result in a split that is not dependent on writer, digit or session. However each writer wrote the exact same digits, 0 - 4, 50 times in a single session thus some of these identical replicates will be in the training set and others in the test set leading to severe spatial-temporal data leakage. Moreover, the performance metrics become heavily over-optimistic as the model simply memorizes specific pixel configurations it has already seen, rather than learning generalizable human writing styles.

2. Split by Individual Writer

In contrast, splitting by individual eliminates leakage as certain writers will never appear in the training pool. For example writers 1- 16 in the training pool and writers 17 - 20 in the test pool. This split is optimal for the Siamese network as it gives a non-optimistic view of verification - to authenticate if two completely unknown hands belong to the same person. However this split breaks the CNN classifier as the model cannot predict or classify the output for writer 17 with only having the output nodes for 1-16.

3. Split by Digit

Similarly, splitting the data by digit will result in the CNN model breaking. While training the model will learn a specific combination of strokes tied to specific character shapes however when presented with a new digit in test it will fail as the geometric structure of the number is foreign. However the Siamese network successfully handles a digit split as it bypasses discrete classification entirely and will take on the general style from the digits in training and apply that information to the test data.

4. Justification for the Session-Based Split

Overall, splitting the train and test data by session is the optimal strategy. This approach provides an identical, perfectly balanced evaluation ground for both the Siamese network and CNN. he CNN retains access to all 20 output classes, while the Siamese network can fairly evaluate similarity metrics across all 5 available digits. Furthermore, there is no optimistic bias or data leakage as session 5 is kept separate from the training loop. Lastly this strategy is optimal as it will be able to handle natural style drift over time (e.g., hand fatigue, mood changes, or stylus wear between sessions) by being trained on historical data (sessions 1 -3), validated on session 4, and tested on future data (Session 5) . This evaluation strategy represents a measure of the Temporal Style Generalisation within a Closed-Universe System. Thus by splitting by session all writers and all digits are preserved across both models.

```
split_df <- data.frame(
  Split = factor(c("Train (60%)", "Validation (20%)", "Test (20%)"),
                 levels = rev(c("Train (60%)", "Validation (20%)", "Test (20%)"))),
  Count = c(15000, 5000, 5000)
)
ggplot(split_df, aes(y = Split, x = Count, fill = Split)) +
  geom_col(width = 0.5) +
  scale_fill_manual(values = c("Train (60%)" = "blue",
                               "Validation (20%)" = "lightblue",
                               "Test (20%)" = "darkblue")) +
  theme_minimal(base_size = 14) +
  labs(x = "Total Image Count", y = NULL) +
  theme(legend.position = "none", panel.grid.major.y = element_blank())
```

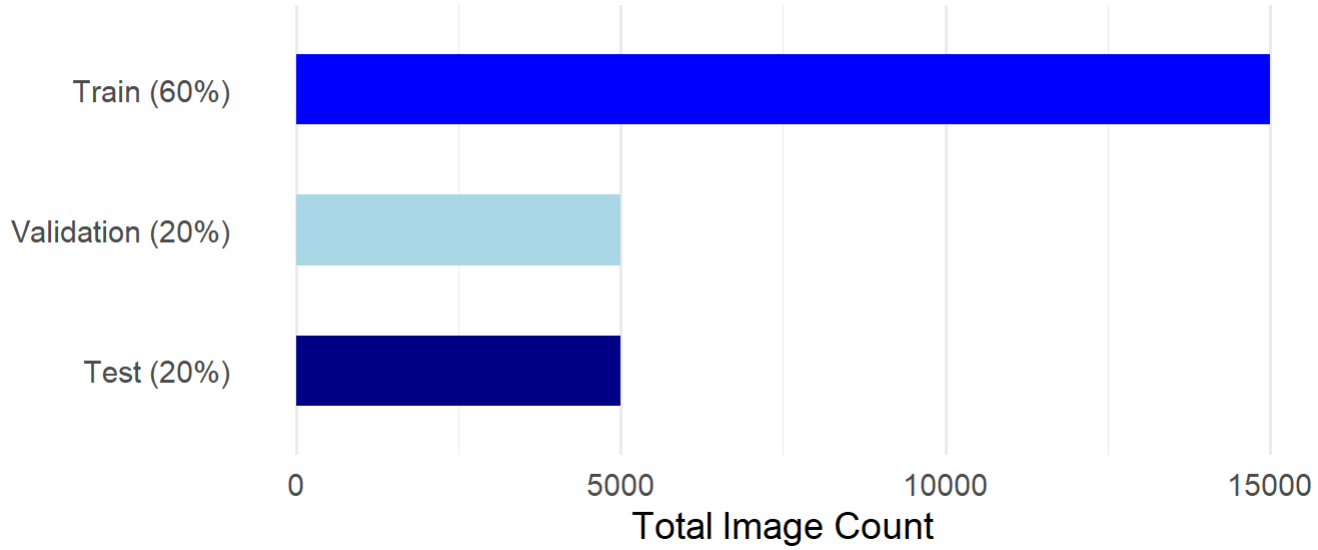


Figure 10: Chronological session partition blueprint

Section 6: Comparing the CNN and Siamese Network

6.1 Overall comparison

```
kable(data.frame(
  ` ` = c("Task", "Question answered", "Chance level", "Main result", "Other results"),
  CNN = c("Identification",
          "Which of the 20 writers wrote this image?",
          "5% (1 in 20)",
          paste0("Accuracy ", pct(cnn_accuracy)),
          paste0("Cohen's kappa ", f3(unname(cnn_cm$overall["Kappa"])))),
  `Siamese network` = c("Verification",
                        "Were these two images written by the same person?",
                        "50% accuracy, AUC 0.5",
                        paste0("AUC ", f3(auc), ", accuracy ", pct(acc)),
                        paste0("True match rate ", pct(tpr), ", false match rate ", pct(fpr)))
```

```
  check.names = FALSE
), row.names = FALSE)
```

Table 5: Test-set (session 5) results from Sections 2 and 3.

	CNN	Siamese network
Task	Identification	Verification
Question answered	Which of the 20 writers wrote this image?	Were these two images written by the same person?
Chance level	5% (1 in 20)	50% accuracy, AUC 0.5
Main result	Accuracy 64.8%	AUC 0.975, accuracy 92.0%
Other results	Cohen's kappa 0.630	True match rate 91.6%, false match rate 7.6%

The two models solve **different tasks**, so their headline numbers cannot be compared directly. The CNN performs identification: from a single image, it must pick the writer out of 20 people. Random guessing would succeed only 5% of the time. The Siamese network performs verification: given two images, it only has to answer "same person" or "different people", where random guessing already succeeds 50% of the time. A Siamese accuracy of 92.0% is therefore not evidence that it is a better model than a CNN with 64.8% accuracy; it was asked an easier question.

Each result is best judged against its own chance level. The CNN's 64.8% is about 13 times better than guessing among 20 writers, and its kappa of 0.630 confirms substantial agreement beyond chance. The Siamese network's AUC of 0.975 is close to the maximum of 1, meaning that in almost every comparison a same-writer pair received a smaller distance than a different-writer pair.

Beyond the numbers, the two approaches suit different real-world uses:

- **Closed group vs new people.** The CNN can only choose among the 20 people it was trained on. Given a stranger's handwriting, it will still confidently name one of the 20. The Siamese network compares samples, so a new person can be added by storing a reference sample, with no retraining.
- **Fixed answer vs adjustable decision.** The CNN gives one answer. The Siamese network gives a distance, so the cut-off can be moved depending on whether false matches or false rejections are more costly.
- **How errors are spread.** The CNN's errors were concentrated: writer 5 was never identified correctly and writer 10 only 24.8% of the time (Section 2). A system that fails completely for particular people can be unusable for them, even when overall accuracy looks reasonable.

6.2 Which digits carry more writer information?

To compare digits, each model is measured on its own task:

- **CNN:** writer identification accuracy for test images of each digit (from Section 2).
- **Siamese network:** verification AUC for test pairs in which **both** images show that digit. This measures how well the writer can be recognised from that digit alone.

```
kable(data.frame(
  Digit = digit_tbl$digit,
  `CNN accuracy` = sprintf("%s ± %.1f", pct(digit_tbl$cnn_acc), 100 * digit_tbl$cnn_ci),
```

```
  `CNN rank` = rank(-digit_tbl$cnn_acc),
  `Siamese AUC (same digit)` = f3(digit_tbl$siam_auc),
  `Siamese rank` = rank(-digit_tbl$siam_auc),
  check.names = FALSE
), row.names = FALSE)
```

Table 6: Writer information by digit (session 5). CNN: identification accuracy with 95% interval. Siamese: verification AUC for pairs where both images show that digit.

Digit	CNN accuracy	CNN rank	Siamese AUC (same digit)	Siamese rank
0	65.3% ± 3.0	2	0.997	1
1	63.3% ± 3.0	4	0.995	4
2	61.3% ± 3.0	5	0.996	3
3	69.6% ± 2.9	1	0.997	2
4	64.6% ± 3.0	3	0.994	5

```
digit_long <- rbind(
  data.frame(digit = digit_tbl$digit, value = digit_tbl$cnn_acc,
             measure = "CNN: identification accuracy"),
  data.frame(digit = digit_tbl$digit, value = digit_tbl$siam_auc,
             measure = "Siamese: verification AUC (same digit)")
)
ggplot(digit_long, aes(x = factor(digit), y = value)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = f3(value)), vjust = -0.3, size = 3) +
  facet_wrap(~ measure, scales = "free_y") +
  labs(x = "Digit", y = "")
```

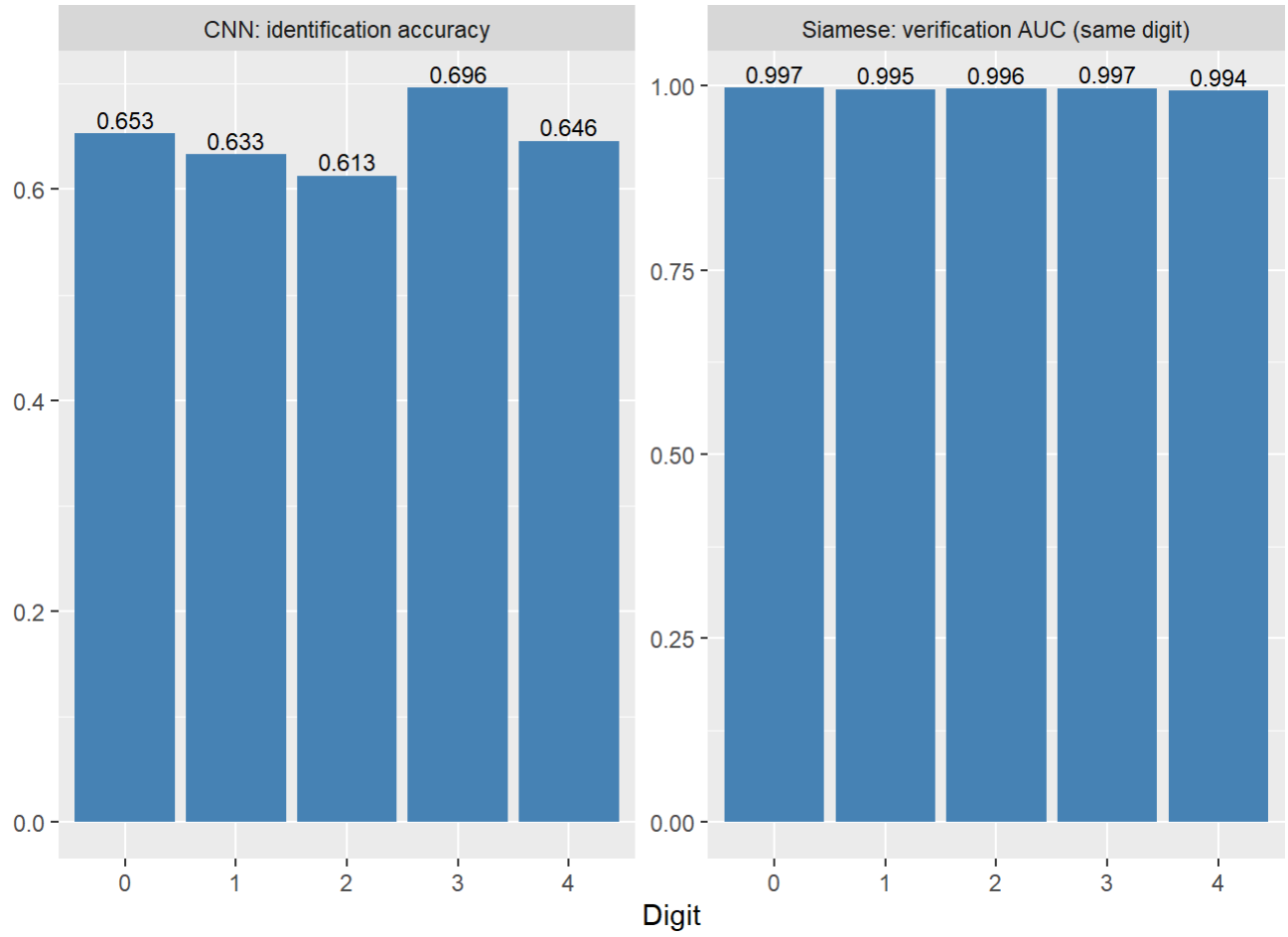


Figure 11: Writer information by digit for each model, on its own task (session 5).

For the CNN, the writer was easiest to identify from digit 3 and hardest from digit 2. For the Siamese network, same-digit matching was easiest for digit 0 and hardest for digit 4. The rank correlation between the two models' digit results was 0.500.

Comparing the two models is what makes this a critical test. If a digit genuinely carries more information about who wrote it, **both** models should find it easier, even though they were trained differently and solve different tasks. Here the two models only partly agree, so any ranking of digits should be treated as tentative.

The size of the differences also matters. With 1,000 test images per digit, each CNN accuracy has a 95% interval of about ±3 percentage points, so only differences larger than about 6 points can be treated as real. The CNN's largest gap (8.3 points) passes this test, but most neighbouring digits do not differ meaningfully. The Siamese AUCs are all close to 1 (range 0.994 to 0.997), so for verification every digit carries enough information when two images of the same digit are compared.

Where differences do appear, they have a plausible explanation. Digits with curves and loops give a writer more room to show personal style: loop size, curvature, how strokes join. A 1 is close to a single straight line, so there is less to distinguish writers beyond slant and thickness.

A clearer pattern appears when the two images show **different** digits. The Siamese AUC averaged 0.996 for same-digit pairs but 0.968 for different-digit pairs. Much of the writer information is therefore **digit-specific**: it is easier to tell whether two 3s share a writer than whether a 3 and a 0 do. When the digits differ, the model has to rely on style that carries across shapes, such as slant, stroke thickness and size, and that is weaker evidence. So the answer to "which digit carries the most writer information" depends on whether the comparison is made within one digit or across digits.

6.3 How session changes affect performance

```
kable(data.frame(
  Session = session_tbl$session,
  Role = session_tbl$role,
  `CNN accuracy` = pct(session_tbl$cnn_acc),
  `Siamese AUC (both images from this session)` = f3(session_tbl$siam_auc),
  check.names = FALSE
), row.names = FALSE)
```

Table 7: Performance in each recording session, each model on its own task. Sessions 1-3 were used for training.

Session	Role	CNN accuracy	Siamese AUC (both images from this session)
1	Training	91.0%	0.995
2	Training	90.5%	0.996
3	Training	95.4%	0.996
4	Validation	60.0%	0.976
5	Test	64.8%	0.973

```
sess_long <- rbind(
  data.frame(session = 1:5, value = session_tbl$cnn_acc,  measure = "CNN: identification accur
  data.frame(session = 1:5, value = session_tbl$siam_auc, measure = "Siamese: verification AUC
)
ggplot(sess_long, aes(x = session, y = value)) +
  annotate("rect", xmin = 0.5, xmax = 3.5, ymin = -Inf, ymax = Inf, alpha = 0.1) +
  geom_line(colour = "steelblue4") + geom_point(size = 3, colour = "steelblue4") +
  facet_wrap(~ measure, scales = "free_y") +
  labs(x = "Session", y = "")
```

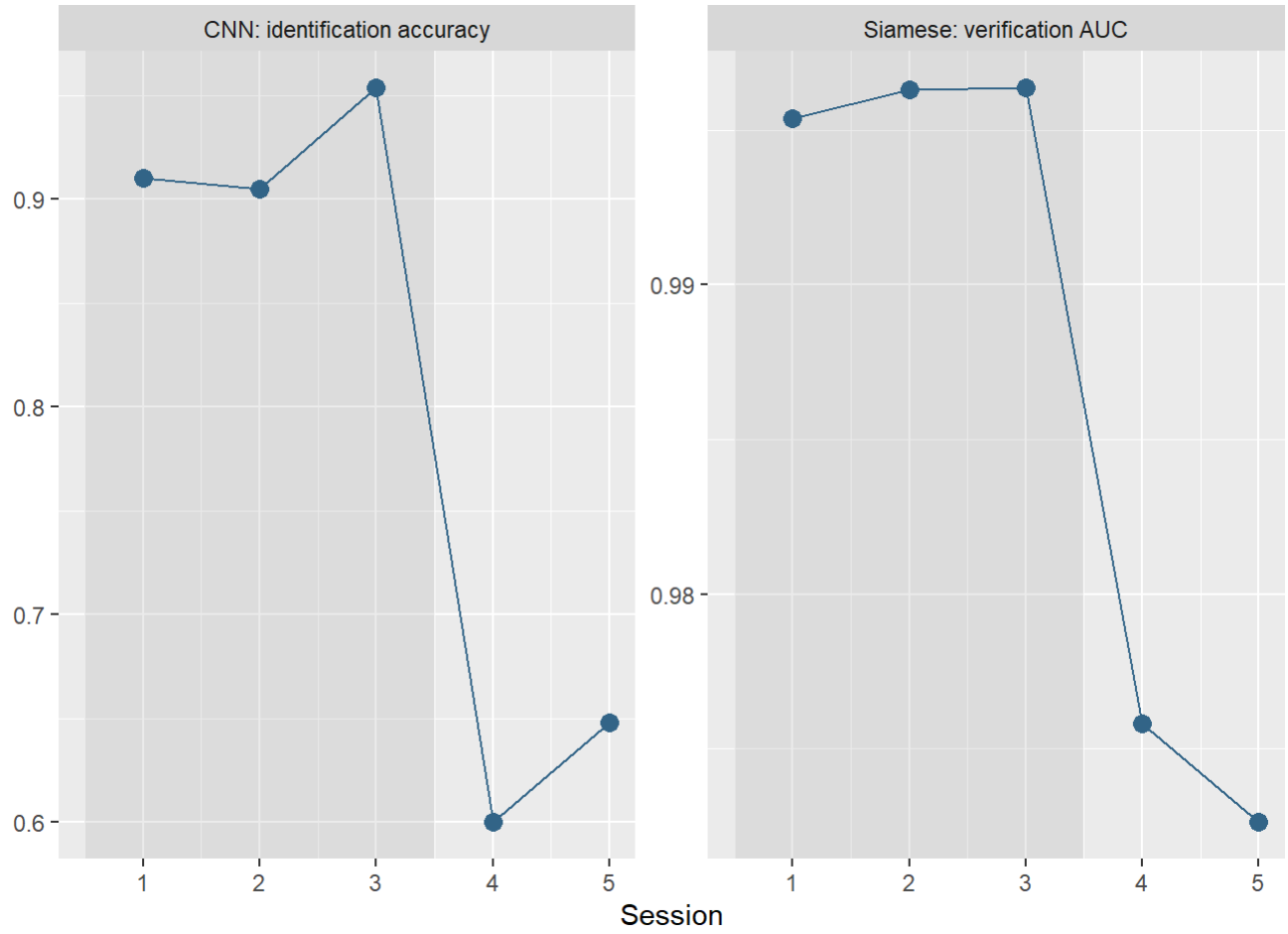


Figure 12: Performance by session for each model on its own task. The shaded area marks the training sessions.

The CNN is strongly affected by session changes. On the sessions it was trained on (1–3) it identified the writer 92.3% of the time; on the two new sessions (4–5) this fell to 62.4%, a drop of 29.9 percentage points. Sessions 4 and 5 performed similarly (60.0% and 64.8%). The loss therefore comes from moving to **any** unseen session, not from session 5 being unusual. The CNN partly learned what each writer's handwriting looked like in those particular training sessions, rather than only the stable features that carry over. This matches the overfitting seen in Section 2, where training accuracy kept rising while validation accuracy levelled off.

The Siamese network is much less affected. Its AUC was 0.996 on average for the training sessions and 0.974 for the new sessions, a change of only 0.022. Because it was trained to compare pairs rather than memorise 20 classes, it learned a more general notion of "same writer" that transfers to new sessions.

```
kable(data.frame(
  Pairs = c("Both images from session 5 (as in Section 3)",
            "Session 5 image vs session 4 image",
            "Session 5 image vs reference from sessions 1-3"),
  `Siamese AUC` = f3(c(within_5, cross_54, cross_513)),
  check.names = FALSE
), row.names = FALSE)
```

Table 8: Siamese AUC depending on which sessions the two images come from.

Pairs	Siamese AUC
Both images from session 5 (as in Section 3)	0.973

Pairs	Siamese AUC
Session 5 image vs session 4 image	0.937
Session 5 image vs reference from sessions 1-3	0.963

In Section 3, both images in every test pair came from session 5. In practice, a new sample is usually compared with a reference collected **earlier**, as with a signature on file. When a session-5 image was compared with a reference from sessions 1–3, the AUC was 0.963, compared with 0.973 when both came from session 5. Comparing across sessions is harder, because the reference and the new sample differ by the session change as well as by normal variation. The within-session result in Section 3 is therefore an optimistic estimate of real-world verification performance.

There is a further reason to be careful with within-session pairs. Section 1 showed that replicates within a session are almost identical, so two images from the same writer and session can be near-copies. That makes within-session matching artificially easy, and the cross-session result is the more realistic measure.

```
kable(data.frame(
  Writer = top_drop$writer,
  `Accuracy, sessions 1-3` = pct(top_drop$acc_seen),
  `Accuracy, session 5` = pct(top_drop$acc_s5),
  `Most often predicted as (session 5)` = top_drop$most_predicted_as,
  check.names = FALSE
), row.names = FALSE)
```

Table 9: CNN: the five writers whose identification accuracy dropped most between the training sessions and session 5.

Writer	Accuracy, sessions 1-3	Accuracy, session 5	Most often predicted as (session 5)
5	98.7%	0.0%	13
10	100.0%	24.8%	2
13	74.8%	0.0%	8
8	98.9%	50.4%	8
16	97.9%	54.4%	16

Session changes did not affect all writers equally (Table 9). For 8 of the 20 writers, the CNN's accuracy fell by more than 30 percentage points from the training sessions to session 5, while others were barely affected. For the worst cases, the CNN did not just become uncertain; it consistently assigned the writer's session-5 images to **one other** writer (last column). This suggests that, for these people, the session change shifted their handwriting towards another writer's style. Writer 5's images being assigned to writer 13 (Section 2) is an example. A classifier with a fixed set of 20 classes is especially vulnerable to this: when a writer's style shifts, their images fall into a neighbour's class.

6.4 Summary

- The CNN (identification, 64.8%, chance 5%) and the Siamese network (verification, AUC 0.975, chance 0.5) answer different questions, so their headline numbers cannot be ranked directly. The

Siamese approach has practical advantages: it can handle new writers and its decision cut-off can be adjusted.

- Digit-level differences were modest. Only differences of more than about 6 percentage points in CNN accuracy can be trusted, and the two models did not fully agree on which digits were easiest. The clearest finding is that writer information is strongest **within** a digit: matching two different digits was noticeably harder for the Siamese network.
- Session changes were the main source of error for the CNN, which lost 29.9 percentage points on unseen sessions, with failures concentrated on a few writers whose style shifted towards someone else's. The Siamese network was far more robust to session changes. Its Section 3 result, based on pairs from a single session, is still somewhat optimistic compared with matching against an earlier reference.
- All results come from simulated handwriting, one held-out test session and a single training run of each model. Small differences should not be over-interpreted.